

Hard disk failure data analysis

Luca Falasca

0334722

luca.falasca@students.uniroma2.eu

Matteo Conti

0323728

matteo.conti97@students.uniroma2.eu

Abstract—Questo studio presenta l’implementazione di una pipeline per lo stream processing di dati di grandi dimensioni, finalizzata all’analisi dei fallimenti dei dischi rigidi. La pipeline, containerizzata e gestita tramite Docker Compose, utilizza Apache Kafka per l’ingestion dei dati e la pubblicazione dell’output del processamento ed Apache Flink per il processamento. Il dataset analizzato, una versione ridotta di quello del Grand Challenge della conferenza ACM DEBS 2024, contiene informazioni su data di misurazione, identificativo del disco rigido, modello, ore di accensione e fallimenti. Per analizzare i dati e confrontare le prestazioni dei formati di file CSV e Parquet sono state eseguite tre query. **TODO**

I. INTRODUZIONE

A. Descrizione del problema

Il problema da affrontare consiste nell’eseguire stream processing di un dataset mediante l’uso di una pipeline basata su framework per Big Data simulando l’arrivo in real time dei dati. Nello specifico, l’obiettivo è analizzare un dataset contenente informazioni relative ai fallimenti dei dischi rigidi attraverso l’esecuzione di 3 query.

B. Obiettivi

L’obiettivo del progetto è quello di eseguire le query proposte su Apache Flink e di valutare le prestazioni ottenute dalle stesse in termini di latenza e throughput.

C. Dataset

Il dataset fornito è una versione ridotta di quello presentato nel Grand Challenge della conferenza ACM DEBS 2024. Delle numerose colonne presenti nel dataset, ne verranno selezionate solamente sette per l’esecuzione delle query, in particolare:

- *date*: data della misurazione
- *serial_number*: identificativo del disco rigido
- *failure*: indica se il disco rigido ha avuto una failure o meno
- *model*: modello del disco rigido
- *vault_id*: identificativo del gruppo di storage server
- *s9_power_on_hours*: ore di accensione del disco rigido
- *s194_temperature_celsius*: temperatura del disco rigido in gradi celsius

Esplorando il dataset si può notare un’ambiguità nella colonna *s9_power_on_hours*, essa contiene dei valori 0. Questi valori possono essere interpretati in due modi:

- Il dato è un errore di misurazione e quindi va eliminato
- Il disco rigido è stato acceso per meno di un’ora al momento della misurazione

Date le seguenti considerazioni si è deciso di non eliminare i valori 0 e considerarli quindi validi. Questo andrà ad influenzare i risultati della query 3, in particolare questo sarà molto evidente per quanto riguarda il minimo delle ore di accensione. Le colonne *s9_power_on_hours* e *s194_temperature_celsius* contiene anche diversi valori nulli, le righe corrispondenti a tali valori verranno eliminate in fase di preprocessamento. Inoltre, righe di header ripetute ad esempio righe in cui il campo *vault_id* non è un intero bensì la stringa "vault_id", anche questa volta tali righe verranno eliminate in fase di preprocessamento.

II. PIPELINE

Per la gestione dei dati relativi ai guasti dei dischi rigidi, è stata implementata una pipeline (Fig. 1) il cui deploy è stato fatto tramite Docker Compose. La pipeline è composta da diverse componenti, ognuna delle quali svolge un compito specifico, in particolare è stato utilizzato Apache Kafka come sistema publish subscriber per la data ingestion e la scrittura dell’output ed Apache Flink per lo stream processing. Di seguito verranno illustrati i dettagli di ciascuna componente.



Fig. 1. Pipeline ad alto livello

A. Data Ingestion

Per quanto riguarda la data ingestion è stato sviluppato un programma che simula il replay del dataset in maniera accelerata. Il replay viene accelerato di un fattore 3600, in modo tale che ogni secondo di tempo reale corrisponda ad un’ora di tempo simulato. Tuttavia dato che il dataset contiene delle tuple a granularità di giornaliera, per evitare di inviare i dati a burst ogni giorno, i dati della giornata vengono spalmati uniformemente all’interno di essa. Per farlo ogni giornata è stata logicamente divisa in 150000 parti, un numero che è stato scelto appositamente maggiore del numero di tuple presenti in ogni giornata, e ad ogni parte che dura 1/150000 di giorno, corrisponde un invio di una tupla al sistema. Per fare in modo che il tempo sia scandito nel modo corretto, viene calcolato a priori il tempo relativo all’invio della prima tupla e poi viene misurato quando effettivamente viene inviata, a quel punto viene fatta una sleep del tempo rimanente a riempire lo slot. Inoltre, dato che le tuple non sono esattamente 150000 per

ogni giornata, gli slot non vengono riempiti mai tutti, e quindi alla fine della giornata viene fatta una sleep per riempire il tempo rimanente fino alla fine della giornata simulata. I dati vengono pubblicati su Kafka in un topic dedicato che poi sarà consumato da Flink.

B. Data Stream Processing

Per il processamento streaming dei dati, e quindi l'esecuzione delle query, è stato utilizzato il framework Apache Flink, in particolare utilizzando la libreria PyFlink. È stato utilizzato un cluster composto da uno job manager ed un task manager dotato di un task slot (Fig. 2). Flink consuma i dati da Kafka, filtrati, processati e successivamente pubblicati nuovamente in un topic Kafka. Per effettuare le query sono state utilizzate le API dei Datastream.

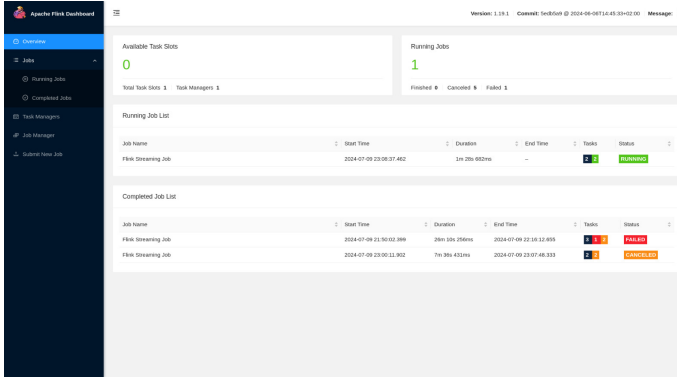


Fig. 2. Cluster Flink

1) *Query 1*: La prima query prevede di calcolare, per i vault (campo *vault_id*) con identificativo compreso tra 1000 e 1020, il numero di eventi valutati, il valor medio e la deviazione standard della temperatura misurata sui suoi hard disk (campo *s194_temperature_celsius*). Per prima cosa vengono generate la source ed il sink Kafka e successivamente, tramite una map, dalle tuple in ingresso vengono estratte le colonne di interesse nel caso di righe del dataset valide e vengono generate delle tuple *Invalid CSV line* per le righe del dataset non valide, al passo successivo tali tuple vengono filtrate. Dopo la fase di filtraggio viene definito come vengono generati i watermark e le tuple vengono marchiate con un timestamp basato sul campo *date* convertito in millisecondi, ricevuto il timestamp vengono filtrate le tuple con *vault_id* compreso tra 1000 e 1020. Sulle rimanenti tuple viene fatto keyBy sul campo *vault_id* considerando finestra di un giorno, tre giorni o globale (chiusa tramite trigger basato sul process time) a seconda della configurazione; le tuple nella finestra vengono aggregate utilizzando una funzione che si occupa di calcolare il numero di eventi valutati, il valor medio e la varianza della temperatura misurata sui suoi hard disk. Infine al risultato viene applicata una map per convertire la varianza in deviazione standard e successivamente viene scritto nel topic Kafka di output relativo alla prima query. Il calcolo avviene tramite una realizzazione di *AggregateFunction* la quale permette di definire un accumulatore i cui campi possono essere aggiornati ad ogni

tupla in arrivo, nel nostro caso l'accumulatore è composto da 5 campi cioè la data della prima tupla relativa ad un certo vault id ricevuta, il vault id della tupla corrente, un contatore che aumenta di 1 ad ogni tupla ricevuta, e due campi che si occupano di implementare l'*algoritmo di Welford* [1] per il calcolo online di media e varianza. In particolare per il calcolo di media e varianza sono state utilizzate le seguenti relazioni:

$$\begin{aligned}\bar{x}_{k+1} &= \bar{x}_k + \frac{x_{k+1} - \bar{x}_k}{k+1} \\ \sigma_{k+1}^2 &= \sigma_k^2 + \frac{(x_{k+1} - \bar{x}_k)(x_{k+1} - \bar{x}_{k+1})}{k+1}\end{aligned}\quad (1)$$

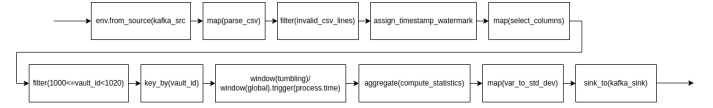


Fig. 3. Dag di alto livello della query 1

2) *Query 2*: La seconda query prevede di calcolare la classifica aggiornata in tempo reale dei 10 vault che registrano il più alto numero di fallimenti nella stessa giornata. Per ogni vault va riportato il numero di fallimenti ed il modello e numero seriale degli hard disk guasti. Per prima cosa vengono generate la source ed il sink Kafka e successivamente, tramite una map, dalle tuple in ingresso vengono estratte le colonne di interesse nel caso di righe del dataset valide e vengono generate delle tuple *Invalid CSV line* per le righe del dataset non valide, al passo successivo tali tuple vengono filtrate. Dopo la fase di filtraggio viene definito come vengono generati i watermark e le tuple vengono marchiate con un timestamp basato sul campo *date* convertito in millisecondi, assegnato il timestamp sulle tuple viene fatto keyBy sul campo *vault_id* considerando finestra di un giorno, tre giorni o globale (chiusa tramite trigger basato sul process time) a seconda della configurazione. Le tuple nella finestra vengono aggregate tramite un *AggregateFunction* che si occupa, tramite accumulatore, di mantenere la data della prima tupla della finestra e, per ogni vault id, calcolare il numero di fallimenti e mantenere una lista contenente il modello ed il numero seriale degli hard disk guasti. All'output dell'aggregazione viene racchiuso in una finestra con una *window_all* applicata una *ProcessAllWindowFunction* che si occupa di calcolare la classifica, questo viene fatto ordinando tutte le tuple nella finestra sul campo che riporta il numero di failure e prendendo le prime 10, l'utilizzo di *window_all* e della *ProcessAllWindowFunction* è stata necessaria per poter effettuare l'ordinamento sull'intero contenuto della finestra. Infine il risultato viene scritto nel topic Kafka di output relativo alla seconda query.

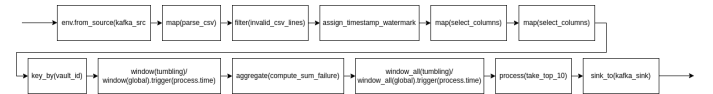


Fig. 4. Dag di alto livello della query 2

3) *Query 3*: La terza query prevede di calcolare il minimo, 25-esimo, 50-esimo, 75-esimo percentile e massimo delle ore di funzionamento (campo *s9_power_on_hours*) degli hark disk per i vault con identificativo tra 1090 e 1120 (compresi) in real time, senza accumulare tutti i valori. Per prima cosa vengono generate la source ed il sink Kafka e successivamente, tramite una map, dalle tuple in ingresso vengono estratte le colonne di interesse nel caso di righe del dataset valide e vengono generate delle tuple *Invalid CSV line* per le righe del dataset non valide, al passo successivo tali tuple vengono filtrate. Dopo la fase di filtraggio viene definito come vengono generati i watermark e le tuple vengono marchiate con un timestamp basato sul campo *date* convertito in millisecondi, ricevuto il timestamp vengono filtrate le tuple con *vault_id* compreso tra 1090 e 1120. Sulle rimanenti tuple viene fatto keyBy sul campo *vault_id* considerando finestra di un giorno, tre giorni o globale (chiusa tramite trigger basato sul process time) a seconda della configurazione; Le tuple nella finestra vengono aggregate tramite un *AggregateFunction* che si occupa, tramite accumulatore, di mantenere la data della prima tupla della finestra e, per ogni vault id, calcolare il minimo, 25-esimo, 50-esimo, 75-esimo percentile e massimo delle ore di funzionamento tramite l'algoritmo *PSquare* [2]. In particolare ad ogni tupla ricevuta vengono aggiornati gli accumulatori relativi ad i valori del PSquare e vengono confrontati i valori di *s9_power_on_hours* con i valori di min e max precedenti ed aggiornarli se necessario, in questo modo i percentili sono stimati ma mentre il minimo ed il massimo sono esatti pur non mantenendo tutti i valori. Infine il risultato viene scritto nel topic Kafka di output relativo alla terza query.

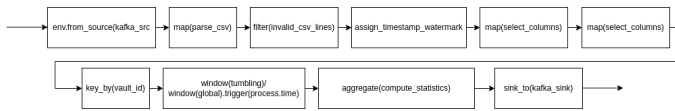


Fig. 5. Dag di alto livello della query 3

C. Data Output

Per la scrittura dell'output del processamento è stato utilizzato Apache Kafka, in particolare sono stati creati tre topic, uno per ogni query, in cui vengono scritti i risultati delle stesse. I topic sono stati creati tramite un'apposita API di Kafka e vengono consumati da un'applicazione esterna che si occupa di scrivere i risultati su file in formato CSV.

D. Analisi delle prestazioni

1) *Metriche*: Le metriche misurate per le 3 query sono la latenza ed il throughput.

- Per la latenza è stata utilizzata una metrica custom tramite l'oggetto Gauge offerto da Flink che fornisce metriche on demand
- Per il throughput è stata utilizzata la metrica built-in di Flink che fornisce il throughput in elementi al secondo NumRecordsOutPerSecond

Entrambe queste metriche sono disponibili tramite l'interfaccia web di Flink e sono accessibili tramite REST API. Le suddette

metriche sono state campionate ad intervalli regolari di 1 secondo tramite uno script Python che fa richieste al job manager utilizzando le REST API di Flink e salva i risultati in un file CSV per poi graficarli.

2) *Query 1*: Si può notare come, per la prima query, le performance di tutte le finestre siano abbastanza in linea tra loro per quanto riguarda il throughput, presentando un andamento piuttosto regolare. Invece per quanto riguarda la latenza essa va a comparire sempre più tardi mano a mano che la finestra aumenta, questo probabilmente è dovuto al fatto che essa viene calcolata solo quando la finestra viene chiusa e quindi per finestre più grandi la latenza risulta essere maggiore perchè viene sperimentato un'attesa maggiore per la chiusura della stessa; infatti è anche possibile vedere che all'aumentare della taglia della finistra le variazioni avvengono con meno frequenza, fino ad avere latenza costante nella finestra globale

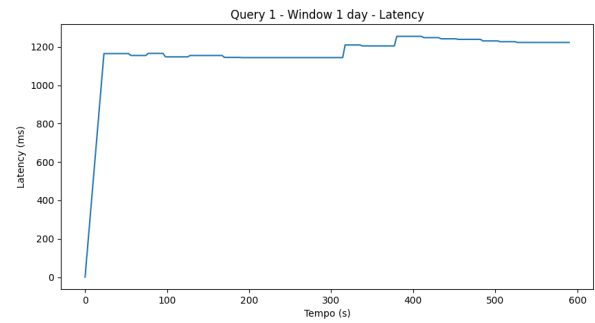


Fig. 6. Latency Query 1 - Window 1 day

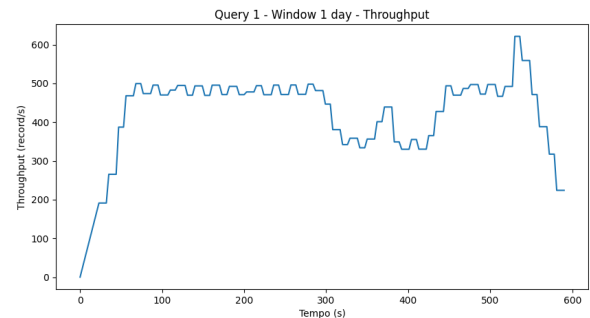


Fig. 7. Throughput Query 1 - Window 1 day

3) *Query 2*: Anche per la query 2 valgono le stesse considerazioni della query1.

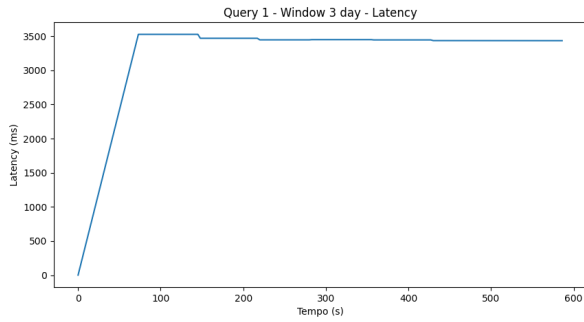


Fig. 8. Latency Query 1 - Window 3 day

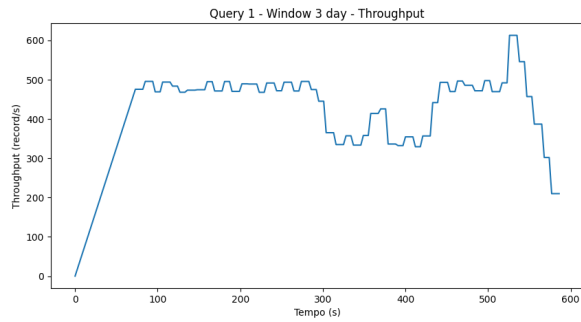


Fig. 9. Throughput Query 1 - Window 3 day

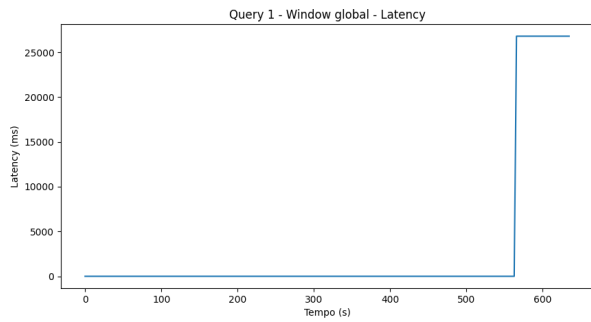


Fig. 10. Latency Query 1 - Window Global

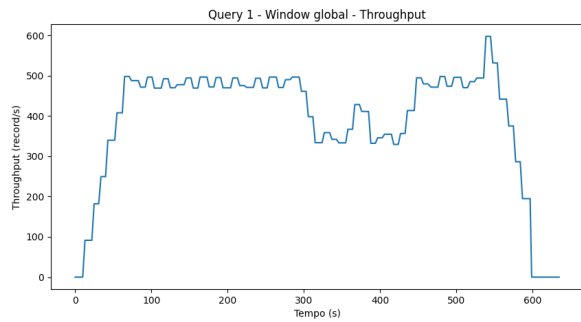


Fig. 11. Throughput Query 1 - Window Global

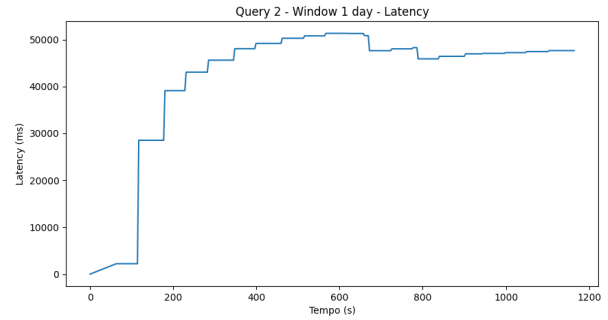


Fig. 12. Latency Query 2 - Window 1 day

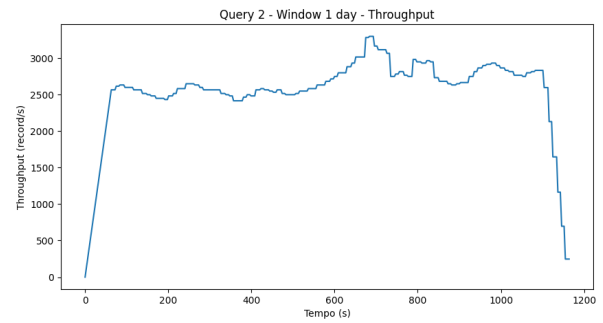


Fig. 13. Throughput Query 2 - Window 1 day

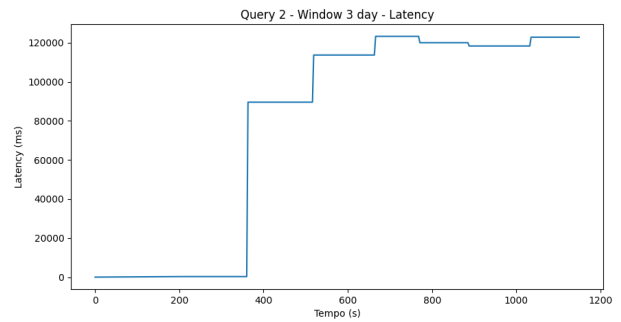


Fig. 14. Latency Query 2 - Window 3 day

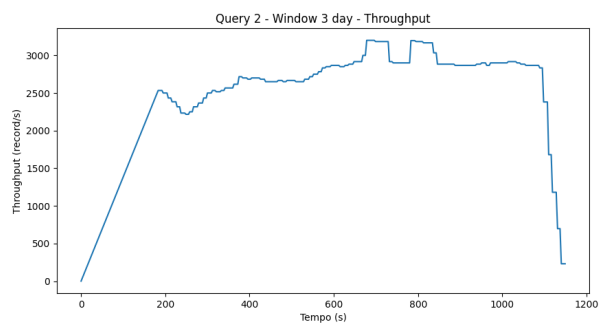


Fig. 15. Throughput Query 2 - Window 3 day

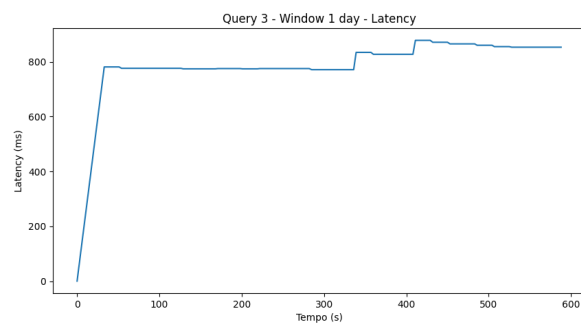


Fig. 18. Latency Query 3 - Window 1 day

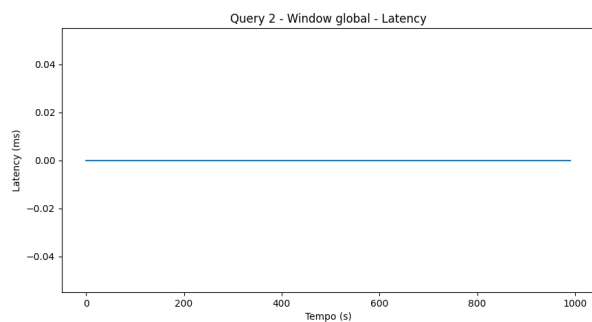


Fig. 16. Latency Query 2 - Window Global

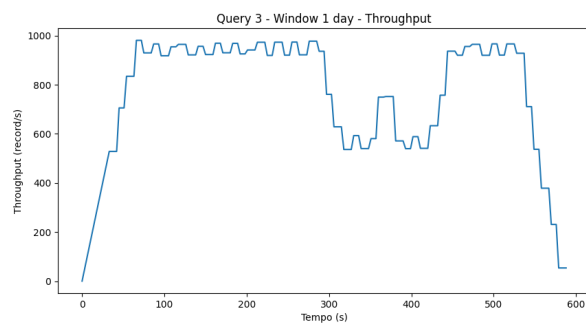


Fig. 19. Throughput Query 3 - Window 1 day

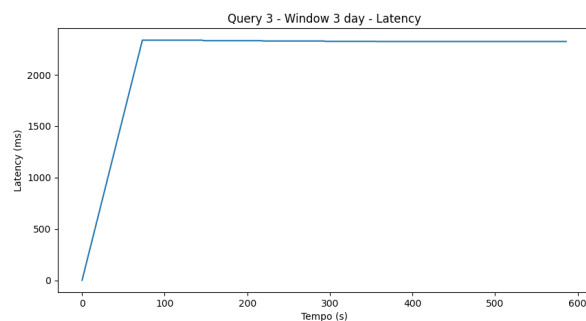


Fig. 20. Latency Query 3 - Window 3 day

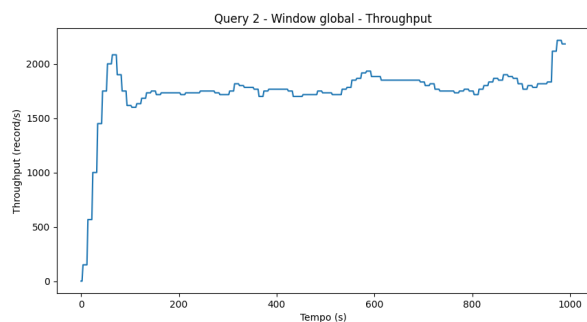


Fig. 17. Throughput Query 2 - Window Global

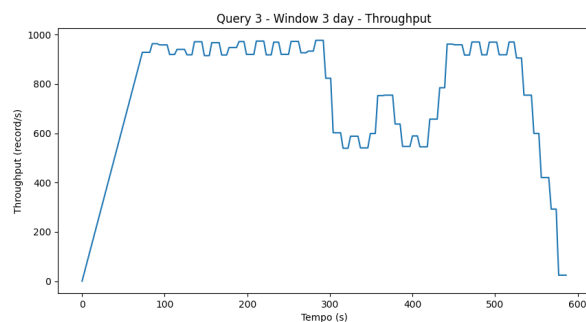


Fig. 21. Throughput Query 3 - Window 3 day

4) *Query 3*: Anche per la query 3 valgono le stesse considerazioni della query1.

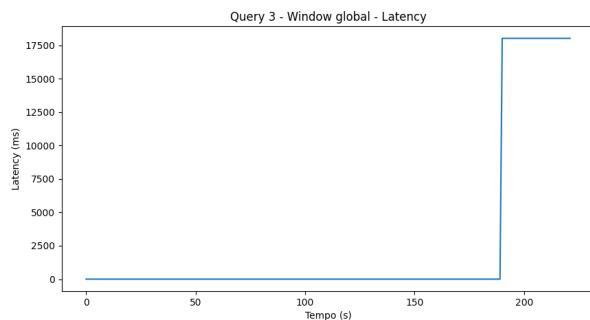


Fig. 22. Latency Query 3 - Window Global

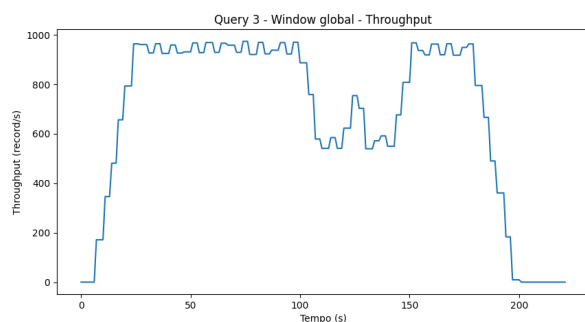


Fig. 23. Throughput Query 3 - Window Global

5) Configurazione ambiente:

Risorse hardware

Processors: AMD Ryzen™ 5 7530U with Radeon™ Graphics × 12
 Memory: 16.0 GiB of RAM
 Graphics Processor: AMD Radeon™ Graphics

Risorse Docker

Processors: 6 core
 Memory: 4 GiB of RAM

Risorse Flink

Task Managers: 1
 Task Slots: 1

6) Risultati:

III. RIFERIMENTI

- [Codice sorgente github](#)

REFERENCES

- [1] R. Jain, I. Chlamtac. "The P^2 Algorithm for Dynamic Calculation of Quantiles and Histograms Without Storing Observations."
- [2] Welford's algorithm https://en.m.wikipedia.org/wiki/Algorithms_for_calculating_variance#Welford%27s_online_algorithm